

C/C++ Secure Code Analyzer

Automated Compiler Audit & Static Vulnerability Report

Target File:	source_code.cpp
Language Mode:	C++ Subset Mode
Student Name:	Engineering Student
Project Guide:	Prof. Project Guide
Institution:	Department of Computer Engineering
Generated Date:	2026-08-30 15:44:22

1. Executive Analysis Summary

Metric	Value
Source File	source_code.cpp
Language Mode	C++ Subset Mode
Lines of Code (LOC)	45
Total Lexical Tokens	114
Semantic Errors	0
Security Findings	11
Security Score	8 / 100 (Poor)

2. Compiler Engine Results

Phase	Status Summary
Lexical Analysis	Passed (114 Tokens)
Syntax Analysis	Passed
AST Generation	Generated Successfully
Semantic Analysis	PASSED (0 Errors)

3. Sample Lexical Token Table

Token Type	Lexeme Value	Line Number
INT	int	6
MAIN	main	6
LPAREN	(	6
RPAREN	)	6
LBRACE	{	6
STRING	string	8
ID	password	8
ASSIGN	=	8
STRING_LITERAL	secret_password_123	8
SEMI	;	8

4. Symbol Table Overview

Identifier	Kind / Type	Scope	Pos
print	BuiltinFunctionSymbol (void)	global	0:0
printf	BuiltinFunctionSymbol (int)	global	0:0
scanf	BuiltinFunctionSymbol (int)	global	0:0
db_query	BuiltinFunctionSymbol (int)	global	0:0
system	BuiltinFunctionSymbol (int)	global	0:0

5. Semantic Diagnostic Errors

No semantic errors detected in program.

6. Static Security Findings

Severity	Rule ID	Vulnerability	Line	Recommendation
Medium	SEC010	Resource Leak (Unclosed File)	40	Ensure every opened file descriptor is explicitly released with close().
High	SEC001	Hardcoded Password	8	Store passwords securely using environment variables or a secret vault.
High	SEC001	Hardcoded Password	9	Store passwords securely using environment variables or a secret vault.
Medium	SEC003	Weak Password	9	Enforce high-entropy password requirements and multi-factor authentication.
High	SEC002	Hardcoded API Key	12	Do not commit API tokens in source code. Use secure secret management APIs.
High	SEC006	Unsafe Function Call (gets)	20	Replace gets() with bounded input alternatives like fgets().
Medium	SEC007	Unsafe Function Call (strcpy)	23	Replace strcpy() with strncpy() or snprintf() providing explicit buffer limits.
Medium	SEC008	Unsafe Function Call (sprintf)	26	Use snprintf() to specify buffer capacity limits explicitly.
High	SEC004	Potential SQL Injection	30	Use parameterized prepared statements instead of dynamic SQL string building.
Critical	SEC005	Potential Command Injection	34	Avoid system() execution. Use safe library APIs with strict argument sanitization.
Low	SEC009	Weak Random Number Generator	37	Use secure_rand() or cryptographically secure PRNGs.

7. AI Educational Explanations

Finding [SEC010] Resource Leak (Unclosed File)

[Offline Fallback Explanation]

1. Vulnerability Name

Resource Leak (Unclosed File)

2. Simple Explanation

System resources allocated via `open()` are not released with matching `close()` calls.

3. Technical Explanation

Static analysis identified `open()` file descriptor handles lacking explicit `close()` cleanup in execution paths.

4. Why it is Dangerous

Accumulating unclosed file descriptors exhausts operating system handle limits, causing application denial of service.

5. Possible Attack Scenario

Repeatedly calling file opening workflows without closing handles consumes all available system file descriptors.

6. Severity Rating

MEDIUM (Resource Exhaustion / Denial of Service)

7. Secure Coding Recommendation

Ensure every opened handle is released using `close()` or managed with RAII resource guards.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
int fd = open("file.txt", 0);
// ... process file ...
close(fd);
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-775: Missing Release of File Descriptor or Resource / OWASP A08:2021

Finding [SEC001] Hardcoded Password

[Offline Fallback Explanation]

1. Vulnerability Name

Hardcoded Password

2. Simple Explanation

Sensitive password credentials are written directly into the source code as plain text.

3. Technical Explanation

Static analysis detected hardcoded credential string literals assigned to variables or constants during AST inspection.

4. Why it is Dangerous

Anyone with access to the codebase or compiled binary strings can extract credentials and gain unauthorized access.

5. Possible Attack Scenario

An attacker reviews committed code repositories or binary string dumps to retrieve the hardcoded password and log in as admin.

6. Severity Rating

HIGH (Loss of Confidentiality & Authentication Bypass)

7. Secure Coding Recommendation

Store passwords in encrypted environment variables or external secret vaults (e.g., AWS Secrets Manager).

8. Corrected Code Example

```
```cpp
// Secure Alternative:
string password = getenv("DB_PASSWORD");
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-259: Use of Hardcoded Password / OWASP A07:2021-Identification and Authentication Failures

Finding [SEC001] Hardcoded Password

[Offline Fallback Explanation]

1. Vulnerability Name

Hardcoded Password

2. Simple Explanation

Sensitive password credentials are written directly into the source code as plain text.

3. Technical Explanation

Static analysis detected hardcoded credential string literals assigned to variables or constants during AST inspection.

4. Why it is Dangerous

Anyone with access to the codebase or compiled binary strings can extract credentials and gain unauthorized access.

5. Possible Attack Scenario

An attacker reviews committed code repositories or binary string dumps to retrieve the hardcoded password and log in as admin.

6. Severity Rating

HIGH (Loss of Confidentiality & Authentication Bypass)

7. Secure Coding Recommendation

Store passwords in encrypted environment variables or external secret vaults (e.g., AWS Secrets Manager).

8. Corrected Code Example

```
```cpp
// Secure Alternative:
string password = getenv("DB_PASSWORD");
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-259: Use of Hardcoded Password / OWASP A07:2021-Identification and Authentication Failures

Finding [SEC003] Weak Password

[Offline Fallback Explanation]

1. Vulnerability Name

Weak Password

2. Simple Explanation

Trivial or predictable default passwords (e.g. 'admin123', 'password') are used in source code.

3. Technical Explanation

AST literal inspection matched low-entropy password strings against a known dictionary of default passwords.

4. Why it is Dangerous

Trivial passwords are easily guessed using automated dictionary or brute-force attacks.

5. Possible Attack Scenario

An attacker uses standard default password lists to gain instant administrative control of the application.

6. Severity Rating

MEDIUM (Weak Authentication Security)

7. Secure Coding Recommendation

Enforce high-entropy password requirements, password salting, and secure hash functions.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
string user_pass = prompt_secure_password();
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-521: Weak Password Requirements / OWASP A07:2021

Finding [SEC002] Hardcoded API Key

[Offline Fallback Explanation]

1. Vulnerability Name

Hardcoded API Key

2. Simple Explanation

API keys or authentication tokens are written explicitly into the source code.

3. Technical Explanation

The static analyzer detected string literals containing API token patterns or key identifiers assigned to variables.

4. Why it is Dangerous

Leaked API keys allow unauthorized third parties to access cloud services, consume API quotas, or alter infrastructure.

5. Possible Attack Scenario

Automated secret scanners harvest public code commits within seconds and execute API requests using the stolen credential.

6. Severity Rating

HIGH (Unauthorized Cloud API Access)

7. Secure Coding Recommendation

Load API keys dynamically from secure environment variables or a secret management vault at runtime.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
string api_key = getenv("API_KEY");
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-798: Use of Hardcoded Credentials / OWASP A07:2021

Finding [SEC006] Unsafe Function Call (gets)

[Offline Fallback Explanation]

1. Vulnerability Name

Unsafe Function Call (gets)

2. Simple Explanation

The gets() function reads input into a buffer without enforcing buffer size limits.

3. Technical Explanation

gets() reads stdin until a newline is encountered, completely ignoring destination buffer capacity.

4. Why it is Dangerous

Overflowing buffer capacity corrupts adjacent stack frame memory, enabling arbitrary remote code execution.

5. Possible Attack Scenario

An attacker inputs text exceeding buffer capacity to overwrite function return addresses with shellcode.

6. Severity Rating

HIGH (Stack Buffer Overflow & Code Execution)

7. Secure Coding Recommendation

Completely eliminate gets(). Use bounded input alternatives like fgets().

8. Corrected Code Example

```
```cpp
// Secure Alternative:
fgets(buffer, sizeof(buffer), stdin);
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-242: Use of Inherently Dangerous Function / OWASP A06:2021

Finding [SEC007] Unsafe Function Call (strcpy)

[Offline Fallback Explanation]

1. Vulnerability Name

Unsafe Function Call (strcpy)

2. Simple Explanation

strcpy() copies source strings into destination buffers without verifying buffer capacity limits.

3. Technical Explanation

Unbounded string copying allows source strings longer than destination buffer allocations to trigger buffer overflows.

4. Why it is Dangerous

Overwriting buffer boundaries leads to application crashes, memory corruption, and potential code execution.

5. Possible Attack Scenario

An attacker inputs long strings that overflow destination buffers on the stack.

6. Severity Rating

MEDIUM (Memory Corruption / Buffer Overflow)

7. Secure Coding Recommendation

Replace strcpy() with bounded functions such as strncpy() or snprintf().

8. Corrected Code Example

```
```cpp
// Secure Alternative:
strncpy(dest, src, sizeof(dest) - 1);
dest[sizeof(dest) - 1] = '\0';
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-120: Buffer Copy without Checking Size of Input / OWASP A06:2021

Finding [SEC008] Unsafe Function Call (sprintf)

[Offline Fallback Explanation]

1. Vulnerability Name

Unsafe Function Call (sprintf)

2. Simple Explanation

sprintf() writes formatted output strings into target buffers without checking output size boundaries.

3. Technical Explanation

Unbounded format expansion can produce string lengths exceeding destination buffer allocation.

4. Why it is Dangerous

Buffer overflow corrupts adjacent heap/stack structures and risks stability or arbitrary code execution.

5. Possible Attack Scenario

Crafting expanded string formats overflows target output buffers during runtime string assembly.

6. Severity Rating

MEDIUM (Format String / Buffer Overflow)

7. Secure Coding Recommendation

Replace sprintf() with snprintf() to specify destination buffer capacity limits.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
snprintf(buffer, sizeof(buffer), "%s", src);
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-134: Use of Externally-Controlled Format String / OWASP A06:2021

Finding [SEC004] Potential SQL Injection

[Offline Fallback Explanation]

1. Vulnerability Name

Potential SQL Injection

2. Simple Explanation

Dynamic string concatenation is used to construct database queries with untrusted input.

3. Technical Explanation

db_query() received dynamically concatenated string expressions or unvalidated identifiers in its AST parameters.

4. Why it is Dangerous

Attackers can manipulate query logic to access, modify, or delete database tables beyond intended access limits.

5. Possible Attack Scenario

Passing "" OR '1'='1" in input parameters bypasses authentication checks and dumps customer database tables.

6. Severity Rating

HIGH (Data Exfiltration & Database Compromise)

7. Secure Coding Recommendation

Use parameterized prepared statements or ORM query builders to bind parameters safely.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
db_query_prepared("SELECT * FROM users WHERE id = ?", user_id);
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-89: Improper Neutralization of Special Elements used in an SQL Command / OWASP A03:2021

Finding [SEC005] Potential Command Injection

[Offline Fallback Explanation]

1. Vulnerability Name

Potential Command Injection

2. Simple Explanation

User-controlled input is passed directly to operating system command execution functions.

3. Technical Explanation

system() function invocation received dynamic non-literal variables that may carry unsanitized shell metacharacters.

4. Why it is Dangerous

Attackers append command separators (e.g. ';', '&&') to execute arbitrary operating system commands with host privileges.

5. Possible Attack Scenario

Passing "file.txt; rm -rf /" executes the intended command followed by destructive system deletion commands.

6. Severity Rating

CRITICAL (Arbitrary Operating System Command Execution)

7. Secure Coding Recommendation

Avoid system(). Use specific library APIs (e.g., execve()) with array-passed arguments without invoking a subshell.

8. Corrected Code Example

```
```cpp
// Secure Alternative:
exec_safe_command("ls", "-la");
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-78: Improper Neutralization of Special Elements used in an OS Command / OWASP A03:2021

Finding [SEC009] Weak Random Number Generator

[Offline Fallback Explanation]

1. Vulnerability Name

Weak Random Number Generator

2. Simple Explanation

Standard `rand()` produces deterministic, predictable pseudo-random sequence values.

3. Technical Explanation

`rand()` relies on simple linear congruential algorithms whose internal states are easily reconstructed from short sample outputs.

4. Why it is Dangerous

Predictable random numbers fail security guarantees when used for session IDs, tokens, or encryption keys.

5. Possible Attack Scenario

An attacker predicts subsequent session tokens generated by `rand()` and hijacks user accounts.

6. Severity Rating

LOW (Predictable Random Sequence Generation)

7. Secure Coding Recommendation

Use cryptographically secure random number generators (`secure_rand()`, `/dev/urandom`, or OS CSPRNG APIs).

8. Corrected Code Example

```
```cpp
// Secure Alternative:
int safe_rnd = secure_rand();
```
```

9. Security Best Practices

Enforce strict input sanitization, safe API alternatives, and resource cleanup.

10. Standard References

CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator / OWASP A02:2021

8. Overall Audit Conclusion & Scoring Summary

| Severity Category | Deduction Weight | Findings Count | Total Deduction |
|-------------------|------------------|--------------------|-----------------------------|
| Critical Severity | -20 points each | 1 | -20 |
| High Severity | -10 points each | 5 | -50 |
| Medium Severity | -5 points each | 4 | -20 |
| Low Severity | -2 points each | 1 | -2 |
| Base Score | 100 Points | Total Findings: 11 | Final Score: 8 / 100 (Poor) |

The analyzed source file achieved an overall Security Score of **8 / 100 (Poor)**. Static compiler analysis identified **0** semantic errors and **11** security vulnerabilities.